// DOMINANDO O PARADIGMA

Python Avançado

Orientação a Objetos: Classes, Objetos, Herança, Encapsulamento e Polimorfismo.

Orientação a Objetos (POO)



Modelagem Real

Representa entidades do mundo real em formato de código estruturado.



Reutilização

Evita código duplicado através de herança e componentes modulares.



Manutenibilidade

Facilita a expansão e correção de sistemas complexos e grandes.

Classes e Objetos: A Base

Uma **Classe** é a planta baixa (molde), enquanto o **Objeto** é a instância concreta construída a partir dela.

Atributos: Variáveis que guardam o estado do objeto.

Métodos: Funções que definem o comportamento do objeto.

```
class Carro:
    def __init__(self, modelo, cor):
        self.modelo = modelo
        self.cor = cor

# Criando um objeto
meu_carro = Carro("Sedan", "Prata")
print(meu_carro.modelo)
```

O Construtor e o parâmetro self

O método `__init__` é executado automaticamente sempre que um novo objeto é instanciado.

O `self` representa a instância atual do objeto, permitindo acessar seus próprios atributos e métodos.

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def apresentar(self):
        return f"Olá, sou {self.nome}"
```

Encapsulamento: Protegendo Dados

O encapsulamento restringe o acesso direto aos dados internos de um objeto, garantindo maior segurança e integridade.

Público: Acessível de qualquer lugar.

Privado (__): Restrito ao interior da classe.

```
class ContaBancaria:
    def __init__(self, saldo):
        self.__saldo = saldo # privado

    def get_saldo(self):
        return self.__saldo
```

Herança: Reaproveitando Código

A herança permite que uma nova classe (**filha**) herde atributos e métodos de uma classe existente (**pai**).

Ideal para hierarquias onde diferentes objetos compartilham características comuns.

```
class Animal:
    def falar(self):
        return "Som genérico"

class Gato(Animal):
    def falar(self):
        return "Miau!"
```

Polimorfismo: Múltiplas Formas

Polimorfismo permite que métodos com o mesmo nome se comportem de maneira diferente dependendo do objeto que os invoca.

Garante flexibilidade e padronização na interface de comunicação entre classes.

```
def emitir_som(objeto_animal):
    print(objeto_animal.falar())

gato = Gato()
emitir_som(gato) # Saída: Miau!
```

Métodos Especiais (Dunder)

Métodos "dunder" (double underscore) como `__str__` personalizam o comportamento nativo dos objetos.

Permitem definir como a classe se comporta ao ser impressa em texto ou comparada.

```python
class Livro:
    def __str__(self):
        return "Livro de Python Avançado"

livro = Livro()
print(livro) # Chama __str__ automaticamente
```

Sistema Orientado a Objetos

Combinando atributos de estado e comportamentos dinâmicos em uma arquitetura limpa.

Gerenciamento completo de regras de negócio encapsuladas dentro da própria classe.

```
class Carrinho:
    def __init__(self):
        self.itens = []

    def adicionar(self, item):
        self.itens.append(item)
```

Resumo da Arquitetura POO

Classes & Objetos

Moldes e instâncias para organizar o código.

Pilares Principais

Encapsulamento, Herança e Polimorfismo.

Métodos Mágicos

Personalização avançada de comportamento nativo.

Parabéns pelo Nível Avançado!

Você agora domina Orientação a Objetos e está pronto
para criar sistemas corporativos robustos.

```
class MestrePython(Desenvolvedor): pass
```